

Observabilité des systèmes - Prometheus

Mettre en œuvre une stack d'Observabilité moderne avec Prometheus & Grafana

Environnement : Deux VM Linux (Debian/Ubuntu)

Déploiement de Prometheus et découverte du modèle "Pull"

Objectifs

- Comprendre l'architecture de Prometheus et le fonctionnement de la collecte de métriques par "Pull" (tirage).
- Déployer l'infrastructure de base et explorer l'interface native.

Énoncé & Travail à réaliser

1. Préparation de l'environnement

Sur votre machine virtuelle, créez un répertoire de travail nommé `tp-observabilite`. Vous utiliserez **Docker Compose** pour orchestrer les différents composants de la stack au fil du TP.

2. Configuration initiale de Prometheus

Créez un fichier de configuration minimal nommé `prometheus.yml` :

global:

scrape_interval: 15s # Intervalle par défaut pour la collecte des métriques

scrape_configs:

- job_name: 'prometheus'

static_configs:

- targets: ['localhost:9090']

3. Lancement du service

Créez le fichier `docker-compose.yml` associé pour démarrer Prometheus :

services:

prometheus:

image: prom/prometheus:v3.0.0

container_name: prometheus

volumes:

- ./prometheus.yml:/etc/prometheus/prometheus.yml

ports:

- "9090:9090"

Démarrez le conteneur et vérifiez qu'il est accessible sur le port 9090 de votre VM.

4. Validation et exploration

1. Accédez à l'interface web de Prometheus via votre navigateur (http://<IP_DE_LA_VM>:9090).
2. Allez dans le menu **Status -> Targets**. Quel est l'état de la cible `prometheus` ? Que signifie-t-il ?
3. Dans l'onglet **Graph**, recherchez la métrique `prometheus_http_requests_total`. Exécutez la requête. Quelle structure possède la donnée retournée ?

Collecte des métriques (Système et Applicatives)

Objectifs

- Monitorer les ressources d'un système d'exploitation à l'aide d'un exportateur tiers.
- Instrumenter le code d'une application pour exposer des métriques métiers personnalisées.

Énoncé & Travail à réaliser

Partie A : Monitoring Système avec Node Exporter

Prometheus ne sait pas lire directement les compteurs de performance de Linux. Il faut lui adjoindre un agent : **Node Exporter**.

1. Modifiez votre fichier `docker-compose.yml` pour y ajouter le service `node-exporter` (image : `prom/node-exporter:latest`). Pensez à l'exposer sur le port 9100.
2. Modifiez le fichier `prometheus.yml` pour ajouter un nouveau job nommé `systeme-linux` pointant vers cette nouvelle cible.

3. Redémarrez vos conteneurs. Vérifiez dans l'interface Prometheus que la nouvelle cible est bien détectée et active (*UP*).

Partie B : Instrumentation d'une Application (Métriques Custom)

On vous fournit le code d'une API Python (Flask) très simple. Votre objectif est d'utiliser la bibliothèque cliente officielle Prometheus pour instrumenter cette application.

Code de base fourni (app.py) :

```
import time
import random
from flask import Flask

app = Flask(__name__)

@app.route('/')
def main():
    return "Bienvenue sur l'API de test !"

@app.route('/traitement')
def traitement():
    # Simule un traitement de durée variable
    time.sleep(random.uniform(0.1, 0.8))
    if random.randint(1, 10) == 5:
        return "Erreur Interne", 500
    return "Traitement réussi !"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Travail à faire :

1. Modifiez ce script pour intégrer le client Prometheus (*prometheus_client*).
2. Vous devez exposer un endpoint */metrics* sur l'application.
3. Déclarez et incrémentez deux métriques :
 - Un **Counter** nommé *api_requests_total* qui compte le nombre total de requêtes, avec des *labels* pour le *method*, le *endpoint* et le *status_code*.
 - Un **Histogram** nommé *api_request_duration_seconds* pour mesurer le temps de réponse de la route */traitement*.
4. Ajoutez cette application à votre stack d'observabilité (cible Prometheus sur le port *5000*).

Requêtes PromQL et Tableaux de bord Grafana

Objectifs

- Maîtriser le langage de requête PromQL pour extraire de l'intelligence des données brutes.
- Construire des visualisations dynamiques et professionnelles.

Énoncé & Travail à réaliser

Partie A : Exercices pratiques PromQL

Depuis l'onglet **Graph** de Prometheus, trouvez les formules pour répondre aux besoins suivants (à consigner dans votre rapport) :

1. Calculer le taux moyen de requêtes par seconde reçues par l'application Python sur les 5 dernières minutes (utilisez la fonction `rate`).
2. Calculer le ratio (en pourcentage) de requêtes en erreur (HTTP 500) par rapport au total des requêtes reçues.
3. Extraire la quantité de mémoire RAM disponible (en octets ou Go) sur la VM à partir des métriques de `node_exporter`.

Partie B : Déploiement et configuration de Grafana

1. Ajoutez **Grafana** (`grafana/grafana:latest`) à votre fichier `docker-compose.yml` sur le port `3000`.
2. Connectez-vous à Grafana (`admin / admin`).
3. Allez dans **Connections** -> **Data Sources** et ajoutez votre serveur Prometheus comme source de données.

Partie C : Création du Dashboard

Créez un tableau de bord nommé **"Supervision Infrastructure & App"** contenant :

- **Un panneau de type "Stat"** indiquant si l'application est en ligne (1) ou hors ligne (0) (via la métrique `up`).
- **Un graphique linéaire** affichant l'évolution de la charge CPU de la VM.
- **Un graphique temporel** affichant le nombre de requêtes par seconde sur l'API, ventilé par code statut (200 vs 500).
- **Bonus** : Importez le tableau de bord communautaire officiel pour Node Exporter (ID : `1860`). Analysez la richesse des indicateurs fournis.

Après-midi : Gestion des alertes avec Alertmanager

Objectifs

- Rendre la stack d'observabilité proactive en configurant des seuils critiques.

- Connecter un gestionnaire de notifications et simuler des pannes en conditions réelles.

Énoncé & Travail à réaliser

1. Définition des règles d'alerte

Créez un fichier nommé `alert.rules.yml` dans votre dossier de travail. Ce fichier va contenir les conditions logiques déclenchant les alertes :

YAML

groups:

- name: alertes-infra-et-app

rules:

- alert: InstanceDown

expr: up == 0

for: 1m

labels:

severity: critical

annotations:

summary: "L'instance {{ \$labels.instance }} est injoignable"

- alert: CodeErreurHaut

expr: rate(api_requests_total{status_code="500"}[5m]) > 0.5

for: 30s

labels:

severity: warning

annotations:

summary: "Taux d'erreur HTTP 500 critique sur l'API"

Modifiez votre `prometheus.yml` pour inclure ce fichier de règles (`rule_files`).

2. Déploiement d'Alertmanager

1. Ajoutez le service `alertmanager` (`prom/alertmanager:latest`) dans votre fichier `docker-compose.yml`.
2. Configurez Prometheus pour qu'il envoie ses alertes vers ce nouveau conteneur en complétant la section `alerting` dans `prometheus.yml`.

3. Scénarios de Crash-Test (Validation)

Vous devez valider le bon fonctionnement de votre système d'alerte en provoquant deux incidents distincts :

- **Incident 1 : Panne Système / Applicative** Coupez volontairement le conteneur de votre application Python (`docker compose stop <nom_du_conteneur_app>`).

Suivez le cycle de vie de l'alerte dans l'interface Prometheus (onglets **Alerts**).
Expliquez la différence entre les états **Inactive**, **Pending** et **Firing**.

- **Incident 2 : Stress CPU** Installez l'outil `stress-ng` sur votre VM (ou créez une boucle infinie en Python) pour saturer l'utilisation du processeur à plus de 90%.
Vérifiez qu'une alerte de surcharge se déclenche après le délai configuré.

Livrables attendus pour l'évaluation

Chaque groupe d'étudiants devra rendre une archive contenant :

1. L'ensemble de l'architecture de configuration : `docker-compose.yml`, `prometheus.yml`, `alert.rules.yml` ainsi que le code `app.py` instrumenté.
2. Un fichier `dashboard.json` correspondant à l'export du tableau de bord Grafana créé.
3. Un compte-rendu au format PDF contenant les réponses aux questions et, **obligatoirement**, les captures d'écran des alertes Prometheus passant à l'état **Firing** lors des simulations de pannes.